

BINARY EXPONENTIATION

Use when: computing a^b fast – $O(\log b)$ instead of $O(b)$. One of the most reused templates in CP.

// Core idea: $a^b = (a^2)^{(b/2)}$ if b even, $a * a^{(b-1)}$ if b odd
// halves the exponent each step instead of multiplying one at a time

BASIC (no mod)

```
long long power(long long a, long long b) {
    long long res = 1;
    while (b > 0) {
        if (b & 1) res *= a; // exponent odd: take current a
        a *= a; // square the base
        b >>= 1; // halve the exponent
    }
    return res;
}
```

MODULAR VERSION (links to powMod on Page 13)

```
long long powMod(long long a, long long b, long long mod) {
    long long res = 1; a %= mod;
    while (b > 0) {
        if (b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}
```

RECURSIVE VERSION

```
long long powRec(long long a, long long b, long long mod) {
    if (b == 0) return 1;
    long long half = powRec(a, b/2, mod);
    long long res = half * half % mod;
    if (b & 1) res = res * a % mod;
    return res;
}
```

COMMON APPLICATIONS

`powMod(a, mod-2, mod);` // modular inverse (mod prime) – see Page 13
`powMod(2, n, mod);` // count of subsets, reachable states, etc.
`power(base, n);` // matrix exponentiation uses this same pattern on matrices
Key note: always use long long – intermediate squarings overflow int fast.

===== USEFUL FACTS =====

```
//  $a^0 = 1$  (base case, always)
//  $a^b * a^c = a^{(b+c)}$  (exponent addition rule)
//  $(a^b)^c = a^{(b*c)}$  (exponent multiplication rule)
//  $a^b \bmod p$ , if  $\gcd(a,p)=1$  and  $p$  prime:  $a^{(p-1)} \equiv 1 \pmod p$  – Fermat's Little Theorem
// number of set bits in  $b$  = number of multiplications needed (rest are squarings)
// time complexity:  $O(\log b)$  multiplications total
//  $a^{(-1)} \bmod p == a^{(p-2)} \bmod p$  (modular inverse,  $p$  prime – see Page 13)
//  $b$  in binary determines which squarings get multiplied into result
// example:  $b=13$  ( $1101_2$ )  $\rightarrow a^{13} = a^8 * a^4 * a^1$ 
// overflow risk:  $(a*a)$  can exceed long long if  $a$  is near  $1e9+$  – reduce mod early
// power(a, b) with  $b$  negative is undefined here – handle sign separately if needed
```